ADS MANAGEMENT SYSTEM

Proof of Concept — Complete Test Plan

32 tests — run all before development begins

#	Test	Type	Who	Blocker?	Time
T1	Session token capture in Shopify localStorage	Technical	Store access	Yes	30 min
T2	Redirect page fires n8n webhook with correct payload	Technical	Developer	Yes	2–3 hrs
T3	Zendesk ticket data — WhatsApp, Viber, Messenger, Email	Technical	Zendesk admin	Yes	1 hr
T4	Zoho Books webhook payload contains custom fields	Technical	Developer	Yes	1–2 hrs
T5	Meta CAPI test event fires with EMQ >= 6.0	Technical	Developer	Yes	1–2 hrs
T6	Session token survives the redirect page	Technical	Developer	Yes	30 min
T7	Session token survives multi-page browsing	Technical	Store access	Yes	1 hr
T8	Email token injection and regex extraction	Technical	Developer	No	1 hr
T9	Estimate custom fields transfer to Sales Order	Technical	Zoho Books access	Yes	30 min
T10	n8n tag update does not destroy existing Zendesk tags	Technical	Developer	Yes	1 hr
T11	Multi-brand routing sends to correct systems	Technical	Developer	Yes	1–2 hrs
T12	Safari iOS localStorage behavior	Technical	iPhone	No	30 min
T13	Staff email/phone capture rate in last 50 Sales Orders	Operational	Zoho Books access	Yes	30 min
T14	Actual Sales Order creation process after payment	Operational	Business owner	Yes	1 hr

T15	RFQ keyword audit — real customer language	Operational	Zendesk access	Yes	1 hr
T16	Channel volume distribution per brand	Operational	Zendesk access	No	30 min
T17	Every brand has distinct separate contact channels	Operational	Business owner	Yes	30 min
T18	Staff SOP for untracked inquiry scenarios	Operational	Business owner	Yes	1 hr
T19	Redirect page timing on slow Philippine mobile connection	Technical	Developer	No	30 min
T20	Shopify script does not conflict with other apps in localStorage	Technical	Developer	Yes	1 hr
T21	Shopify script does not slow down page load speed	Technical	Developer	Yes	1 hr
T22	Same customer contacts via two channels — duplicate ticket handling	Technical	Developer	Yes	1 hr
T23	Zendesk ticket creation speed for time-based matching	Technical	Zendesk admin	Yes	30 min
T24	RFQ trigger does not fire on agent replies containing keywords	Technical	Developer	Yes	30 min
T25	n8n downtime — Zoho Books webhook retry and correct processing	Technical	Developer	Yes	1–2 hrs
T26	n8n does not send duplicate CAPI events on webhook retry	Technical	Developer	Yes	1 hr
T27	Cross-brand contamination — same phone contacts two brands	Technical	Developer	Yes	1 hr
T28	n8n data store TTL — identifiers expire without stale matches	Technical	Developer	Yes	1 hr
T29	Sales Order edited after creation — webhook and CAPI behavior	Technical	Developer	Yes	1 hr
T30	Sales Order deleted — n8n handles gracefully	Technical	Developer	No	30 min
T31	Meta CAPI fbc format accepted correctly	Technical	Developer	Yes	30 min
T32	Staff bypassing Zendesk — direct Zoho Books estimate creation	Operational	Business owner	Yes	30 min

Total estimated time: 4–5 days to run all 32 tests. Add 1–5 days to resolve any failures found.
Recommended time before development starts: 2 weeks including fixes and confirmations.
Run T3, T13, T17, T23 first — they require no technical setup and answer the most critical unknowns.
All tests marked Blocker = Yes must pass before development starts. No exceptions.
Tests marked Blocker = No: document the result and note the accepted risk. Do not skip them.

Before you start

These 32 tests verify every assumption the system architecture is built on — both technical and operational. Run them in the order listed. Some later tests depend on setup from earlier ones.

You will need:

- One brand Shopify store with admin access (OS 2.0 not required for most tests)
- Zendesk admin access and Postman (or any API testing tool)
- Zoho Books admin access for one brand organization
- Meta Business Manager admin access with at least one active ad account
- Hetzner VPS with n8n installed (required for T2, T4–T6, T8, T10, T11, T19–T29, T31)
- An iPhone with Safari (T12)
- Personal WhatsApp, Viber, Messenger, and email accounts for sending test messages
- Google PageSpeed Insights or browser DevTools Network tab (T21)
- Two different phone numbers for cross-brand contamination test (T27)

PRIORITY ORDER — run these first, no technical setup needed:
T3 — Send test messages on each channel and inspect Zendesk tickets (1 hour)
T13 — Open last 50 Sales Orders in Zoho Books and check email/phone fields (30 min)
T17 — List all brand contact channels and confirm no overlap (30 min)
T23 — Time how long between sending a Viber/Messenger message and ticket appearing in Zendesk (30 min)
These four tests alone will surface the most critical issues before any developer time is spent.

T1 — Session token capture

T1	Session token capture in Shopify localStorage	Who	Time
		Anyone with store access	30 min

What we are verifying: The tracking script in theme.liquid correctly saves session token, fbclid, and UTM parameters to localStorage when a customer lands on the store.

Steps

1. Add the session token script from document Section 3.7.1 to theme.liquid. Save.
2. Open the store in a browser with this URL: [store].myshopify.com?fbclid=TESTVALUE123&utm_source=facebook&utm_medium=paid_social&utm_campaign=test
3. Open browser developer tools (F12) → Application → Local Storage → [store domain].
4. Check for these keys: hc_session_token, hc_fbclid, hc_utm_source, hc_utm_medium, hc_utm_campaign.

✓ PASS condition:

```
hc_session_token exists with format HC-[13-digit timestamp]-[5-digit number]
hc_fbclid = TESTVALUE123
hc_utm_source = facebook, hc_utm_medium = paid_social
```

✗ FAIL condition and what it means:

```
Keys empty or missing → script syntax error or not saved. Fix and retry.
hc_fbclid empty but token present → getParam() function not reading URL params correctly. Debug the function.
Script skips execution on live store → check Shopify.designMode guard is only blocking the theme editor, not the live store.
```

T2 — Redirect page fires n8n webhook

T2	Redirect page fires n8n webhook with correct payload	Who	Time
		Developer	2–3 hours

What we are verifying: When a customer clicks /wa-go, the redirect page sends a POST webhook to n8n containing session token, fbclid, channel, brand, and timestamp before opening WhatsApp.

Prerequisites

- n8n installed and running on Hetzner (document Section 3.6)
- T1 passed

Steps

1. In n8n, create a workflow with a Webhook node. Method: POST. Path: /test-click. Click Listen for test event. Copy the webhook URL.
2. Create the /wa-go redirect page on Shopify per document Section 3.7.2. Set n8n Webhook URL to the test URL from step 1.
3. Visit the store with ?fbclid=TESTVALUE123. Confirm token is in localStorage (T1).
4. Click the WhatsApp button. WhatsApp should open.
5. In n8n, check the webhook execution. Inspect the full payload.

Expected payload


```
{"session_token":"HC-1721234567890-18291","fbclid":"TESTVALUE123","utm_source":"facebook",
  "utm_medium":"paid_social","utm_campaign":"test","channel":"WhatsApp",
  "brand":"[store name]","timestamp":"2026-07-14T08:00:00.000Z"}
```


✓ PASS condition:

```
n8n execution log shows received payload within 5 seconds
session_token matches localStorage value exactly
fbclid = TESTVALUE123
WhatsApp opened with no visible delay
```

✗ FAIL condition and what it means:

```
n8n receives nothing → fetch() call failing. Check browser console for errors.
```

```
Confirm n8n URL is publicly accessible.
   Payload received but session_token empty → localStorage not being read by
redirect page. Verify hc_ prefix matches exactly.
   WhatsApp does not open → window.location.replace() blocked. Check designMode
guard is not blocking live store execution.
```

T3 — Zendesk ticket data per channel

T3	Zendesk ticket data — WhatsApp, Viber, Messenger, Email	Who	Time
		Zendesk admin + Postman	1 hour

What we are verifying: Each contact channel creates a Zendesk ticket containing a reliable identifier n8n can use to match the ticket to the original click log. Run this test first — no technical setup required.

RUN THIS TEST FIRST. Send real messages on each channel right now and inspect the tickets.
This is the most critical unknown in the entire system.

3A — WhatsApp

1. Send a WhatsApp message to your WhatsApp Business number from a personal phone.
2. Open the resulting Zendesk ticket. Note the requester ID from the ticket URL.
3. In Postman: GET https://[subdomain].zendesk.com/api/v2/users/[requester_id] Authorization: Basic [base64 email/token:api-token]
4. In the response JSON, find the phone field. Record the exact value.

✓ PASS condition:

```
   User object contains a non-empty phone field with the customer WhatsApp number
e.g. +639XXXXXXXXX
```

✗ FAIL condition and what it means:

```
   phone field is null → WhatsApp phone not accessible via Zendesk user API.
Change WhatsApp matching to time-based only. Lower confidence but workable.
```

3B — Viber (Oapps) — most critical

1. Send a Viber message to your Viber Business account.
2. Open the resulting Zendesk ticket. Look at everything: subject, body, custom fields Oapps populated, requester profile, tags.
3. In Postman: GET /api/v2/tickets/[ticket_id] and GET /api/v2/tickets/[ticket_id]/comments — inspect full JSON for any Viber identifier.
4. Also check: GET /api/v2/users/[requester_id]/identities — look for any messaging identity type.

✓ PASS condition:
```
   Ticket contains a Viber user ID or consistent Viber identifier across messages
from the same person
```

✗ FAIL condition and what it means:
```
   No identifier → time-based matching only for Viber first contact. Flag all
Viber attributions as lower confidence.
   Phone number present → better than expected. Update architecture to use phone
number matching for Viber.
```

3C — Messenger

1. Send a Facebook Messenger message to your brand page.
2. In Postman: GET /api/v2/users/[requester_id]/identities — look for identity with type = messaging.
3. Send a second Messenger message from the same account. Confirm PSID is identical across both.

✓ PASS condition:
```
   Identities API returns identity with type = messaging
   PSID is identical for both messages from same person
```

✗ FAIL condition and what it means:
```
   No messaging identity → Zendesk not storing PSID. Time-based matching only for
Messenger.
```

3D — Email

1. Send an email to your sales email address from a personal account.
2. Open the resulting Zendesk ticket. Confirm requester email is visible.
3. Check: is the full email body visible in ticket comments?

✓ PASS condition:
```
   Requester email visible on ticket and in user API response
   Email body visible in ticket comments
```

✗ FAIL condition and what it means:
   Email body stripped by Zendesk → token extraction via regex will not work. Rely on email address matching only.

T4 — Zoho Books webhook payload

T4	Zoho Books webhook payload contains custom fields	Who	Time
		Developer	1–2 hours

What we are verifying: When a Sales Order is created in Zoho Books, the webhook payload sent to n8n contains the custom tracking fields with the values that were entered on the Sales Order.

Steps

1. Create all 8 custom tracking fields in Zoho Books Sales Orders module per document Section 3.10.1.
2. In n8n, create a test webhook to log the incoming payload. Copy the URL.
3. In Zoho Books: Settings → Automation → Workflow Actions → Webhooks → New Webhook. Module: Sales Orders. URL: n8n test URL. Method: POST. Body: Default Payload. Save.
4. Settings → Automation → Workflow Rules → New Rule. Module: Sales Orders. Event Based. Action: Created. Immediate Action: Webhook → select webhook. Save.
5. Create a test Sales Order. Fill in custom fields: Session Token = HC-TEST-12345, FB Click ID = TESTFBCLID123.
6. Check n8n execution log. Inspect the full JSON payload received.

✓ PASS condition:

```
n8n receives webhook within 30 seconds of Sales Order save
Payload contains session_token = HC-TEST-12345 and fbclid = TESTFBCLID123
Payload contains Sales Order total and currency
```

✗ FAIL condition and what it means:

```
Custom fields missing from payload → change Workflow 4 to make a separate GET API call to Zoho Books after receiving the webhook trigger, rather than reading custom fields from payload.
Webhook does not fire → check Workflow Rule is active. Check Zoho Books webhook log for error messages.
Payload empty → change webhook body type from Default to Custom and manually specify fields to include.
```

T5 — Meta CAPI test event

T5	Meta CAPI test event fires with EMQ >= 6.0	Who	Time
		Developer	1–2 hours

What we are verifying: n8n can send a conversion event to Meta CAPI and Meta receives it with Event Match Quality of 6.0 or higher.

Prerequisites

- Meta CAPI Dataset created — Events Manager → Connect Data Sources → Offline → Create
- Dataset ID and Access Token obtained — Events Manager → Dataset → Settings → Conversions API → Generate Access Token
- 90-day attribution window enabled — Events Manager → Dataset → Settings → Attribution

Steps

1. In Meta Events Manager → Dataset → Test Events tab. Copy the Test Event Code.
2. In n8n, create a manual-trigger workflow. Add a Code node:

```
const crypto = require('crypto');
const email = 'your-test-email@gmail.com'.trim().toLowerCase();
const phone = '+639XXXXXXXXX'.replace(/[\s\-()]/g,'');
const hashedEmail = crypto.createHash('sha256').update(email).digest('hex');
const hashedPhone = crypto.createHash('sha256').update(phone).digest('hex');
return [{ json: { hashedEmail, hashedPhone } }];
```

3. Add HTTP Request node after Code node:

```
Method: POST
URL: https://graph.facebook.com/v19.0/[DATASET_ID]/events?access_token=[ACCESS_TOKEN]
Body (JSON):
{ "data": [{ "event_name": "Purchase", "event_time": [unix timestamp],
  "action_source": "system_generated", "event_id": "TEST-001",
  "user_data": { "em": "{{ $json.hashedEmail }}", "ph":
"{{ $json.hashedPhone }}" },
  "custom_data": { "value": 1000, "currency": "PHP" } }],
  "test_event_code": "[YOUR_TEST_EVENT_CODE]" }
```

4. Execute the workflow manually.
5. In Meta Events Manager → Test Events tab → click Detect events. Wait up to 3 minutes.
6. Check the Event Match Quality score.

✓ PASS condition:

Purchase event appears in Test Events within 3 minutes

EMQ score is 6.0 or higher

HTTP Request node shows 200 response with events_received: 1

✗ FAIL condition and what it means:

Event does not appear → check n8n log for API response. Common: wrong Dataset ID, expired token, wrong action_source. Must be exactly 'system_generated'.

EMQ below 5.0 → PII hashing incorrect or phone format wrong. Meta requires E.164: +63XXXXXXXXXX with no spaces.

HTTP 400 → event_time must be Unix timestamp integer not a date string.

HTTP 401 → token expired. Regenerate in Events Manager → Dataset → Settings → Conversions API.

IMPORTANT: Remove test_event_code from production workflow after this test passes.
Test events do not count toward real Meta optimization.

T6 — Token survives redirect page

T6	Session token survives the redirect page	Who	Time
		Developer	30 minutes

What we are verifying: The session token in localStorage when the customer is on the Shopify store is identical to the token that appears in the n8n webhook payload when they click the contact button.

Steps

1. Visit the store with ?fbclid=TESTVALUE123. Record the exact session token from localStorage.
2. Click the WhatsApp button (/wa-go).
3. In n8n, open the received webhook execution. Find the session_token in the payload.
4. Compare: is it character-for-character identical to what you recorded in step 1?

✓ PASS condition:

```
session_token in n8n payload is exactly identical to token from localStorage
```

✗ FAIL condition and what it means:

```
Token different → redirect script generating new token instead of reading existing one. Fix: read localStorage.getItem('hc_session_token') in redirect script instead of generating.
Token empty in payload but present in localStorage → localStorage not accessible from redirect page. Check both pages are on the same domain.
```

T7 — Token survives multi-page browsing

T7	Session token survives multi-page browsing	Who	Time
		Anyone with store access	1 hour

What we are verifying: If a customer lands from a Meta ad, browses 5 product pages, and clicks WhatsApp 15 minutes later, the session token is still the same one generated when they first arrived.

Steps

1. Visit store with ?fbclid=TESTVALUE123. Record session token from localStorage.
2. Navigate to 5 different product pages by clicking around the store naturally.
3. Wait 15 minutes.
4. Check localStorage — is the same token still there?
5. Click the WhatsApp button. Check n8n payload — same token?

✓ PASS condition:

```
Same token in localStorage after 5 pages of browsing
Same token in localStorage after 15 minutes
Same token in n8n payload after clicking WhatsApp
```

✗ FAIL condition and what it means:

```
Token changes after navigating → script generating new token on every page
load. Fix: script must check if hc_session_token already exists before generating.
The document script already includes this — verify it was pasted correctly.
Token disappears after time → check if any Shopify app is clearing
localStorage. Test without third-party apps active.
```

T8 — Email token injection and extraction

T8	Email token injection into mailto and regex extraction	Who	Time
		Developer	1 hour

What we are verifying: The mailto link injects the session token into the pre-populated email body, and n8n can extract it from the Zendesk ticket using a regex pattern.

Steps

1. Add the mailto token injection script from document Section 3.7.2 to theme.liquid. Save.
2. Visit store with ?fbclid=TESTVALUE123. Record the session token.
3. Click the Email Us button. Check the email body in your email client — does it contain: Ref: HC-[token]?
4. Send the email to your sales address.
5. Open the resulting Zendesk ticket. Is the token visible in the ticket comment?
6. In n8n Code node, test the regex:

```
const body = 'Hi, I would like to inquire. Ref: HC-1721234567890-18291';
const match = body.match(/Ref:\s*(HC-[0-9]+-[0-9]+)/i);
const token = match ? match[1] : null;
// Expected: token = 'HC-1721234567890-18291'
```

✓ PASS condition:

```
   Email client shows Ref: HC-[token] in pre-populated body
   Token visible in Zendesk ticket comment after email is sent
   Regex correctly extracts token from ticket body text
```

✗ FAIL condition and what it means:

```
   Email body empty on mobile → iOS Mail and some Android clients strip mailto
body. Accepted limitation. Fall back to email address matching.
   Token in email client but not in Zendesk → Zendesk stripping email body. Check
Zendesk email settings for content filtering rules.
   Regex no match → token format changed. Update regex pattern to match actual
format.
```

T9 — Estimate fields transfer to Sales Order

T9	Estimate custom fields transfer to Sales Order on conversion	Who	Time
		Anyone with Zoho Books access	30 minutes

What we are verifying: When staff convert an Estimate to a Sales Order in Zoho Books, the custom tracking field values carry over automatically.

Steps

1. Confirm all 8 custom fields exist in BOTH Estimates AND Sales Orders modules with exactly matching labels.
2. Create a test Estimate. Fill in: Session Token = HC-TEST-12345, FB Click ID = TESTFBCLID123.
3. Convert Estimate to Sales Order: open Estimate → click Convert → Sales Order.
4. Open the Sales Order. Check the custom fields section.

✓ PASS condition:

```
Sales Order custom fields show HC-TEST-12345 and TESTFBCLID123
All 8 custom field values carried over from the Estimate
```

✗ FAIL condition and what it means:

```
Custom fields empty on Sales Order → field names do not match exactly between modules OR Zoho Books does not auto-transfer custom fields on conversion. Fix: add a step in n8n Workflow 3 to copy custom field values from Estimate to Sales Order via API after conversion.
```

T10 — n8n tag update without destroying existing tags

T10	n8n updates Zendesk tags without destroying existing tags	Who Developer	Time 1 hour

What we are verifying: When n8n adds a tag to a Zendesk ticket, it does not remove any tags already on the ticket. The native n8n Zendesk Update Ticket node replaces all tags — so the HTTP Request node with the /tags endpoint must be used instead.

Steps

1. In Zendesk, create a test ticket manually. Add tags: test_tag_1, test_tag_2, brand_primary.
2. In n8n, build a workflow using HTTP Request node:

Method: PUT
URL: https://[subdomain].zendesk.com/api/v2/tickets/[ticket_id]/tags
Authorization: Basic [base64 email/token:api-token]
Content-Type: application/json
Body: { "tags": ["tracking_enriched"] }

NOTE: PUT /api/v2/tickets/{id}/tags ADDS to existing tags.
This is different from PUT /api/v2/tickets/{id} which REPLACES all tags.
Always use the /tags endpoint for additions.

3. Execute the n8n workflow.
4. Open the Zendesk ticket. Check all tags present.

✓ PASS condition:

```
   Ticket has all four tags: test_tag_1, test_tag_2, brand_primary,
tracking_enriched
   No existing tags were removed
```

✗ FAIL condition and what it means:

```
   Only tracking_enriched remains → wrong endpoint used. URL must be
/api/v2/tickets/{id}/tags not /api/v2/tickets/{id}.
   403 Forbidden → API token lacks permission. Must be generated by a Zendesk
admin account.
```

T11 — Multi-brand routing

T11	Multi-brand routing sends to correct systems per brand	Who	Time
		Developer	1–2 hours

What we are verifying: When n8n receives a webhook with a brand field, it routes to that brand's specific Zoho Books organization and Meta CAPI Dataset — not another brand's systems.

Steps

1. In n8n, create a test workflow with Webhook trigger → Switch node routing on the brand field.
2. Connect Switch output 1 (Primary Brand) to a Set node with Primary Brand org ID and CAPI Dataset ID.
3. Connect Switch output 2 (Brand B) to a Set node with Brand B org ID and CAPI Dataset ID.
4. Send test webhook: { brand: 'Primary Brand', session_token: 'HC-TEST-001' }. Check logged values.
5. Send test webhook: { brand: 'Brand B', session_token: 'HC-TEST-002' }. Check logged values.

✓ PASS condition:

```
Primary Brand webhook → logs Primary Brand org ID and Dataset ID
Brand B webhook → logs Brand B org ID and Dataset ID
No cross-contamination between brands
```

✗ FAIL condition and what it means:

```
Both route to same org ID → Switch node condition not matching. Check brand field name is case-sensitive match.
Brand B routes to Default → Brand B value in payload does not match Switch condition. Check for trailing spaces or capitalisation differences.
```

T12 — Safari iOS localStorage

T12	Safari iOS localStorage behavior on iPhone	Who	Time
		Anyone with iPhone	30 min + 7-day follow-up

What we are verifying: Session token saved in localStorage persists across browser sessions on Safari on iPhone. Safari is the most privacy-restrictive common browser and a significant portion of Philippine traffic.

Steps

1. On iPhone, open Safari. Visit the store with ?fbclid=TESTVALUE123.
2. Confirm hc_session_token is in localStorage (use Eruda mobile console if no Mac available: add eruda.io script to theme.liquid temporarily).
3. Close Safari completely (swipe up in app switcher).
4. Wait 5 minutes. Reopen Safari. Return to store. Check localStorage — same token?
5. Repeat after 7 days.

✓ PASS condition:

```
Token persists after closing and reopening Safari
Token persists after 7 days
```

✗ FAIL condition and what it means:

```
Token gone after closing Safari → Safari ITP clearing localStorage. Accepted risk — document it. For iPhone users who close Safari between browsing and messaging, attribution is lost. Not a blocker.
Token gone after 7 days → Safari 7-day ITP cleanup. Accepted risk. These customers fall back to email/phone matching.
```

T13 — Staff data capture rate

T13	Email and phone capture rate in last 50 Zoho Books Sales Orders	Who	Time
		Anyone with Zoho Books access	30 minutes

What we are verifying: Staff consistently capture customer email addresses and phone numbers in Zoho Books. Meta CAPI event match quality depends entirely on this data being present.

WHY THIS IS CRITICAL: Meta CAPI matches your conversion event to a real Facebook user using hashed email and phone. If these are empty, n8n has nothing to hash. The CAPI event is wasted. A technically perfect system fed by incomplete data will perform poorly. This is a people problem, not a tech problem.

Steps

1. In Zoho Books, go to Sales → Sales Orders. Review the last 50 Sales Orders across all brands.
2. For each Sales Order, check: does the linked customer contact have an email address? A phone number?
3. Count how many of 50 have email. How many have phone. Calculate percentages.
4. Check quality: are emails real (not test@test.com)? Are phones in correct format with country code (+63)?

✓ PASS condition:

```
At least 80% of last 50 Sales Orders have a real customer email address
At least 80% have a phone number with country code
No placeholder or dummy values
```

✗ FAIL condition and what it means:

```
Below 80% email capture → mandatory SOP required before go-live: staff must enter email and phone before estimate is sent. Training issue, not technical.
Phone numbers missing country code → staff SOP: all phones must be +63XXXXXXXXX format.
Below 50% on either → do not start development until this is resolved. System cannot fix incomplete data.
```

T14 — Actual Sales Order creation process

T14	How staff actually handle payment and Sales Order creation	Who	Time
		Business owner or sales manager	1 hour

What we are verifying: Staff convert a paid quotation to a Sales Order in Zoho Books within 24 hours of payment. The CAPI event fires at Sales Order creation — delays mean late or missed attribution.

Questions to ask your sales staff

1. When a customer confirms payment, what is the first thing you do?
2. How long between receiving payment and creating the Sales Order in Zoho Books?
3. Is there any case where a sale is recorded somewhere else first — spreadsheet, WhatsApp, verbal?
4. Do different brands follow different processes?
5. Are Sales Orders ever created before full payment (e.g. on deposit)?

Also check

Review the last 20 Sales Orders. Compare the created_at date vs the invoice date. Is there a consistent gap?

✓ PASS condition:

```
Staff create Sales Order in Zoho Books within 24 hours of payment confirmation
All brands follow the same process
No parallel tracking in spreadsheets or other tools
Sales Orders only created for confirmed full payments
```

✗ FAIL condition and what it means:

```
Gap more than 48 hours → CAPI fires late. Meta attribution window handles up to 62 days but longer gaps weaken the signal. Implement SOP: Sales Order must be created within 24 hours of payment.
Different brands use different processes → document each brand's process. Some may need a different CAPI trigger.
Sales Orders created on deposit → decision needed: trigger CAPI on deposit or full payment? Changes the Zoho Books webhook configuration.
Parallel tracking in spreadsheets → those sales will never trigger CAPI. Estimate what % of sales bypass Zoho Books — this is your permanent attribution
```

```
blind spot.
```

T15 — RFQ keyword audit

T15	What words customers actually use when requesting a quotation	Who	Time
		Anyone with Zendesk access	1 hour

What we are verifying: The RFQ detection trigger keyword list covers the real language your customers use — including Filipino and Taglish — so the automatic Zoho Books estimate creation works for real inquiries.

Steps

1. In Zendesk, filter tickets from last 3 months that resulted in a Sales Order in Zoho Books.
2. Open the first customer message in each of these tickets.
3. List every phrase customers used when asking for a price or quotation.
4. Compare against the current keyword list from the document:

Current keyword list: quote, quotation, rfq, estimate, pricing, how much, magkano

Check: Are there common phrases NOT in this list?
Do customers ask indirectly (e.g. 'what is the price of X' instead of 'can I get a quote')?
Do customers write in Filipino, Taglish, or English?
Are there brand-specific phrases (product name + price)?

5. Add all missing phrases to the Zendesk RFQ trigger keyword list before development starts.

✓ PASS condition:

```
   Existing keyword list covers at least 80% of first messages from customers who
bought
   All additional phrases discovered are added to the trigger list
```

✗ FAIL condition and what it means:

```
   Many buyers used phrases not in keyword list → update list with all discovered
phrases. Common Filipino additions: presyo, pabili, magkano po, pwede bang malaman
ang presyo, gaano kalaki ang bayad.
   Customers primarily write in Filipino → add all Filipino variants before
development starts.
```

T16 — Channel volume distribution

T16	Percentage of inquiries per contact channel	Who	Time
		Anyone with Zendesk access	30 minutes

What we are verifying: The system's overall CAPI attribution quality depends on which channels customers actually use. WhatsApp has the strongest attribution. Viber has the weakest. If most conversions come through Viber, overall attribution quality will be lower than expected.

Steps

1. In Zendesk, filter tickets from last 3 months. Group by channel.
2. Count tickets per channel: WhatsApp, Messenger, Viber, Email, other.
3. Calculate percentage of total for each.
4. Cross-check: of tickets that resulted in a Sales Order, what % came from each channel?

Channel	% of total inquiries	% of inquiries that converted	Attribution confidence
WhatsApp	____%	____%	High — phone number matching
Messenger	____%	____%	Medium-High — PSID matching
Viber	____%	____%	Medium — time-based + Viber ID
Email	____%	____%	High when token present
Other/untagged	____%	____%	None — not tracked

✓ PASS condition:

```
WhatsApp + Messenger + Email account for at least 60% of converting inquiries
Viber below 40% of converting inquiries
Other/untagged below 10%
```

✗ FAIL condition and what it means:

Viber above 60% of converting inquiries → overall CAPI quality lower than expected. Consider actively directing customers to WhatsApp for inquiries.

Other/untagged above 10% → customers using untracked channels. Identify and add them or accept as blind spot.

One channel dominates at 80%+ → simplify architecture to focus on that channel.

T17 — Brand contact channel separation

T17	Every brand has distinct separate contact channels with no overlap	Who Business owner	Time 30 minutes

What we are verifying: The Zendesk auto-tagging system identifies which brand a ticket belongs to by detecting which channel it came from. This only works if no two brands share a WhatsApp number, Messenger page, Viber account, or email address.

Complete this table before development starts

Brand	WhatsApp number	Messenger page URL	Viber bot URI	Inquiry email
Primary Brand				
Brand B				
Brand C				
Brand D				
Brand E				
Brand F				
Brand G				
(add rows)				

✓ PASS condition:

```
Every brand has a unique WhatsApp Business number
Every brand has its own Facebook Page for Messenger
Every brand has its own Viber Business account
Every brand has its own inquiry email address
All channels confirmed as active and working
```

✗ FAIL condition and what it means:

```
Two brands share any channel → auto-tagging cannot distinguish between them.
Each brand must have its own separate channel for that platform before development
starts.
```

Brand not yet launched has no channels → define all channels before Zendesk triggers are built. Adding later requires adding new triggers.

T18 — Staff SOP for untracked inquiries

T18	Staff have a clear process for inquiries arriving outside tracked channels	Who	Time
		Business owner / operations	1 hour

What we are verifying: Not every customer arrives from a Meta ad. Staff need to know exactly what to do with walk-ins, phone calls, referrals, and other untracked inquiries so they do not create false CAPI events or pollute attribution data.

Define a process for each scenario

Scenario	Decision needed	Staff action
Customer calls by phone	Does Zendesk ticket get created? If yes, how?	Define: create manual ticket OR accept as untracked
Customer walks into showroom	How is inquiry recorded?	Define: create manual ticket OR accept as untracked
Customer referred by friend — no ad click	Should CAPI event fire for this sale?	CAPI must NOT fire — no fbclid. Staff leave tracking fields blank.
Customer messages staff personal WhatsApp	Bypasses business number	Define: move to business number OR accept as untracked
Customer messages staff personal Facebook	Bypasses Messenger page	Define: move to official page OR accept as untracked
Repeat customer from 6+ months ago — no current tracking	Original fbclid is long expired	Treat as new untracked inquiry. Never fabricate a session token.

Steps

1. For each scenario above, decide on the correct staff action.
2. Write a one-paragraph SOP for each scenario.
3. Most critical rule to communicate: if there is no session token in the custom field, leave it blank. Staff must NEVER invent or guess a session token.
4. Train all sales staff before system goes live.
5. Add the SOP to your Zendesk agent guide or staff reference materials.

✓ PASS condition:

Written SOP exists for all 6 scenarios

All sales staff briefed on when to leave tracking fields blank

Staff understand CAPI events only fire for customers who arrived via Meta ad click

Process exists for creating manual Zendesk tickets for phone and walk-in inquiries

✗ FAIL condition and what it means:

No SOP → staff make inconsistent decisions. Some will fill in tracking fields for non-ad customers. This creates false CAPI events and corrupts Meta optimization. Write SOP before go-live — non-negotiable.

T19 — Redirect page timing on slow mobile connection

T19	Redirect page timing on slow Philippine mobile connection	Who	Time
		Developer	30 minutes

What we are verifying: The redirect page fires the n8n webhook AND opens WhatsApp with no perceptible delay even on a slow mobile data connection. The webhook call must not block the redirect.

Steps

1. On a real Android or iPhone using mobile data (not WiFi) in the Philippines, open the brand store.
2. Click the WhatsApp button (/wa-go). Use a stopwatch — measure time from click to WhatsApp opening.
3. In n8n, check if the webhook was received.
4. Simulate a slow connection: in Chrome DevTools → Network tab → throttle to 'Slow 3G'. Click /wa-go again. Measure time.

✓ PASS condition:

```
WhatsApp opens within 500ms on mobile data
WhatsApp opens within 2 seconds on throttled Slow 3G
n8n webhook received in both cases (even if slightly delayed)
```

✗ FAIL condition and what it means:

```
WhatsApp takes more than 2 seconds to open → the fetch() call is blocking the redirect. Fix: confirm keepalive: true is set on the fetch call so it fires asynchronously. window.location.replace() must run immediately after fetch() is called, not after it resolves.
n8n webhook not received on slow connection → keepalive fetch may be failing on slow networks. Acceptable risk — log missed webhooks in the daily audit. Not a system blocker.
```

T20 — Script does not conflict with other Shopify apps

T20	Shopify tracking script does not conflict with other apps using localStorage	Who	Time
		Developer	1 hour

What we are verifying: The hc_ prefixed localStorage keys used by the tracking script do not collide with keys used by other installed Shopify apps. A collision could cause another app to overwrite your session token or vice versa.

Steps

1. In browser DevTools → Application → Local Storage — open the brand store WITH the tracking script installed and WITH all other Shopify apps active.
2. List ALL localStorage keys present. Look for any keys starting with hc_ that were not created by your script.
3. Also check: are any other apps reading or clearing localStorage on a timer? Use DevTools → Sources → Event Listener Breakpoints → Storage to detect any storage writes during normal browsing.
4. Check if any existing Shopify app (chat widgets, pop-ups, review apps) also stores a session token or visitor ID in localStorage under a key that might conflict.

✓ PASS condition:

```
No other app uses the hc_ prefix in localStorage
No other app clears or overwrites hc_session_token
Tracking script keys are isolated from all other app keys
```

✗ FAIL condition and what it means:

```
Another app uses hc_ prefix → rename your keys. Choose a more unique prefix
(e.g. hcads_ or a brand-specific prefix). Update the tracking script, redirect
page script, and all n8n workflow payload references.
Another app clears localStorage on a timer → assess how often. If frequent, the
token will be lost mid-session. Consider also storing the token in a first-party
cookie as a backup.
```

T21 — Script does not slow down page load

T21	Shopify tracking script does not slow down page load speed	Who Developer	Time 1 hour

What we are verifying: The tracking script added to theme.liquid does not measurably reduce page load speed. A slow store loses customers before they can be tracked.

Steps

1. Before installing the script: run the brand store URL through Google PageSpeed Insights (pagespeed.web.dev). Record the Performance score and Total Blocking Time.
2. Install the tracking script in theme.liquid. Save.
3. Run the same URL through PageSpeed Insights again. Record the new scores.
4. Compare: did Performance score drop? Did Total Blocking Time increase?
5. Also check in Chrome DevTools → Network tab: does the script add any blocking network requests on page load?

✓ PASS condition:

```
   PageSpeed Performance score does not drop by more than 2 points after script
installation
   Total Blocking Time does not increase by more than 50ms
   Script adds no blocking network requests — the fetch() to n8n is async and non-
blocking
```

✗ FAIL condition and what it means:

```
   Performance drops significantly → the script is blocking page render. Fix: add
async or defer attribute considerations. The script is already wrapped in an IIFE
but confirm it is placed before </head> not in <body>. Also confirm the fetch()
call uses keepalive and does not await.
   fetch() to n8n is blocking → this should never happen with the document script
design. If it does, move the fetch call to after window.location.replace().
```

T22 — Duplicate ticket handling

T22	Same customer contacts via two channels — duplicate Zendesk ticket handling	Who	Time
		Developer	1 hour

What we are verifying: When the same customer sends messages on two different channels for the same inquiry (e.g. WhatsApp and then Messenger), n8n correctly handles both tickets without creating duplicate Zoho Books estimates or sending duplicate CAPI events.

Steps

1. Visit the brand store with ?fbclid=TESTVALUE123. Note the session token.
2. Click the WhatsApp button. Send a message. Confirm Zendesk ticket is created and enriched with token.
3. From the same browsing session, click the Messenger button. Send a message. Confirm second Zendesk ticket is created.
4. Check: does n8n enrich the Messenger ticket with the same session token as the WhatsApp ticket?
5. In the WhatsApp ticket, type a message containing 'quote'. Check: does Zoho Books create one estimate or two?
6. If two estimates are created: does n8n have deduplication logic to prevent this?

✓ PASS condition:

```
   Both tickets enriched with the same session token
   Only one Zoho Books estimate created — not one per channel
   If second estimate creation is attempted, n8n detects duplicate by session
token and skips creation
```

✗ FAIL condition and what it means:

```
   Two estimates created with same token → add deduplication logic to Workflow 3:
before creating an estimate, check if one already exists for this session token.
If yes, skip creation and add internal note to ticket linking to existing
estimate.
   Messenger ticket not enriched → time-based matching window may have elapsed.
Review matching window timing.
```

T23 — Zendesk ticket creation speed

T23	Time gap between customer sending Viber/Messenger message and Zendesk ticket appearing	Who Zendesk admin	Time 30 minutes

What we are verifying: The time-based matching in n8n for Viber and Messenger assumes the Zendesk ticket appears within 30 minutes of the customer clicking the redirect page. This test measures the actual gap to confirm the 30-minute window is sufficient.

Steps

1. Record the exact time on your phone clock (to the second).
2. Send a Viber message to your Viber Business account. Record the send time.
3. Immediately watch Zendesk. Record the exact time the ticket appears.
4. Calculate the gap in seconds.
5. Repeat 5 times at different times of day. Record all 5 gaps.
6. Repeat the same test with Messenger. Record all 5 gaps.
7. Identify the maximum gap observed across all 10 tests.

Test #	Channel	Message sent time	Ticket appeared time	Gap (seconds)
1	Viber			
2	Viber			
3	Viber			
4	Viber			
5	Viber			
6	Messenger			
7	Messenger			
8	Messenger			
9	Messenger			

10	Messenger			

✓ PASS condition:

Maximum gap across all 10 tests is under 10 minutes

The 30-minute matching window in n8n comfortably covers the worst-case delay

✗ FAIL condition and what it means:

Any gap exceeds 15 minutes → increase the n8n matching window to 60 minutes. More than 60 minutes is a sign Oapps or Zendesk integration is unreliable — escalate to Oapps support.

Gaps are inconsistent (sometimes 30 seconds, sometimes 20 minutes) → time-based matching is unreliable for this channel. Lower confidence. Flag all Viber/Messenger attributions from time-based matches as uncertain.

T24 — RFQ trigger does not fire on agent replies

T24	RFQ trigger does not fire when an agent reply contains a quotation keyword	Who Developer	Time 30 minutes

What we are verifying: The Zendesk RFQ detection trigger only fires on customer messages, not on agent replies. If an agent writes 'I am sending you a quote now', the trigger must not fire again and create a duplicate Zoho Books estimate.

Steps

1. Open any existing Zendesk ticket.
2. As an agent, add a public reply containing the word 'quote': 'Thank you, I am preparing your quote now.'
3. Check: does the rfq_detected tag get added to the ticket?
4. Check: does a new Zoho Books estimate get created?

✓ PASS condition:

```
No rfq_detected tag added on agent reply
No Zoho Books estimate created from agent reply
```

✗ FAIL condition and what it means:

```
rfq_detected fires on agent reply → the Zendesk trigger condition is missing
the requester filter. Fix: add condition 'Current user is Requester' OR 'Role is
not Agent' to the trigger conditions. This ensures the trigger only fires when the
ticket requester (the customer) adds a comment, not when an agent replies.
Duplicate estimate created → add the requester filter above AND add
deduplication logic in Workflow 3 (check for existing estimate before creating).
```

T25 — n8n downtime and webhook retry

T25	n8n downtime — Zoho Books webhook retry and correct processing	Who	Time
		Developer	1–2 hours

What we are verifying: When n8n is temporarily unavailable and Zoho Books fires a Sales Order webhook, Zoho Books retries the webhook after n8n comes back online. n8n processes the retry correctly and sends the CAPI event exactly once.

Steps

1. Stop n8n: on the Hetzner server, run: sudo docker compose stop n8n
2. In Zoho Books, create a test Sales Order with custom fields filled in.
3. Wait 5 minutes. Confirm Zoho Books attempted the webhook (check Settings → Automation → Webhooks → webhook log for failed attempts).
4. Restart n8n: sudo docker compose start n8n
5. Wait for Zoho Books to retry (Zoho Books retries failed webhooks on its own schedule).
6. Check n8n execution log — did the webhook eventually arrive and process correctly?
7. Check Meta Test Events — did exactly one CAPI Purchase event fire?

✓ PASS condition:

```
Zoho Books webhook log shows failed attempt during n8n downtime
After n8n restart, webhook is retried and received
n8n processes it correctly — CAPI event fires
Only one CAPI event fires — not duplicated
```

✗ FAIL condition and what it means:

```
Zoho Books does not retry → Zoho Books webhook retry policy is not active or
max retries already exhausted. Check Zoho Books webhook settings for retry
configuration. If no retry: implement a scheduled n8n workflow that polls Zoho
Books daily for Sales Orders created in the last 24 hours that have not been sent
to CAPI.
Two CAPI events fire → deduplication on event_id not working. See T26.
```

T26 — No duplicate CAPI events on retry

T26	n8n does not send duplicate CAPI events when a webhook is received more than once	Who Developer	Time 1 hour

What we are verifying: If Zoho Books retries a webhook (or n8n processes the same Sales Order webhook twice for any reason), Meta only receives one CAPI Purchase event — not two.

How Meta deduplication works

Meta deduplicates CAPI events using the event_id field within a 48-hour window. The system uses SO-[salesorder_id] as the event_id. Meta will drop any event with an event_id it has already processed within 48 hours.

Steps

1. In n8n Workflow 4, confirm the event_id in the CAPI payload is set to: 'SO-' + salesorder_id
2. Send the same CAPI Purchase event twice with identical event_id. Use the manual test workflow from T5.
3. Use the same event_id both times: event_id = 'SO-TEST-DEDUP-001'
4. In Meta Events Manager → Test Events tab, check how many events appear.
5. Wait 10 minutes and check again.

✓ PASS condition:

```
   Only one Purchase event appears in Meta Test Events despite sending the same
event_id twice
   Meta deduplication is working correctly
```

✗ FAIL condition and what it means:

```
   Two events appear → Meta deduplication is not catching the duplicate. Check
that event_id is exactly identical in both payloads — character for character.
Also check that both events are sent to the same Dataset ID. If deduplication
still fails: add an n8n data store check — store a record of processed
salesorder_ids and skip CAPI send if the ID was already processed.
```

T27 — Cross-brand customer contamination

T27	Same customer phone number contacts two different brands — n8n routes correctly	Who Developer	Time 1 hour

What we are verifying: When the same customer (same phone number) contacts Brand A and then later contacts Brand B, n8n correctly creates separate session token mappings for each brand and does not apply Brand A's token to Brand B's inquiry.

Steps

1. Using Phone Number 1: visit Primary Brand store with ?fbclid=FBCLID_BRAND_A. Click /wa-go. Note session token A.
2. Send a WhatsApp message to Primary Brand's WhatsApp number. Confirm Zendesk ticket is enriched with token A and brand = Primary Brand.
3. Using the same Phone Number 1: visit Brand B store (different URL) with ?fbclid=FBCLID_BRAND_B. Click Brand B's /wa-go. Note session token B — it should be a different token.
4. Send a WhatsApp message to Brand B's WhatsApp number. Check the resulting Zendesk ticket.
5. Confirm: ticket is tagged brand = Brand B AND session_token = token B (not token A).

✓ PASS condition:

```
Brand B ticket has brand = Brand B
Brand B ticket has session_token = token B (from the Brand B store visit)
Brand A's session token is not applied to the Brand B ticket
n8n data store has two separate entries for Phone Number 1 — one per brand
```

✗ FAIL condition and what it means:

```
Brand B ticket gets token A → n8n data store is matching on phone number alone
without checking brand. Fix: change the data store key from 'whatsapp_[phone]' to
'whatsapp_[brand]_[phone]' so each brand has its own phone→token mapping. This
prevents cross-brand contamination.
```

T28 — n8n data store TTL

T28	n8n data store identifiers expire correctly and do not cause stale matches	Who	Time
		Developer	1 hour

What we are verifying: Customer identifiers (phone numbers, PSIDs, Viber IDs) stored in the n8n data store for repeat-contact matching expire after the configured TTL (365 days) and do not cause stale or incorrect matches for new contacts from the same customer a year later.

Steps

1. In n8n, open the data store used for storing identifier → session token mappings.
2. Manually create a test entry: key = 'whatsapp_primary_+639111111111', value = 'HC-OLD-TOKEN-99999', TTL = 1 minute.
3. Wait 2 minutes.
4. Try to retrieve the key from the data store. Confirm it no longer exists.
5. Now create a new contact from the same phone number. Confirm n8n correctly creates a new session token mapping rather than retrieving the expired one.

✓ PASS condition:

```
   Entry with 1-minute TTL is gone after 2 minutes
   n8n creates a fresh session token mapping for the same phone number after TTL
expiry
   No stale matches from expired entries
```

✗ FAIL condition and what it means:

```
   TTL not working — entry still present after expiry → check n8n data store TTL
configuration. Some n8n versions handle TTL differently. If TTL is unreliable,
implement a scheduled cleanup workflow that purges data store entries older than
365 days.
```

T29 — Sales Order edited after creation

T29	Sales Order edited after creation — webhook behavior and duplicate CAPI prevention	Who	Time
		Developer	1 hour

What we are verifying: When staff edit a Sales Order after it is created (e.g. to correct the amount), Zoho Books may fire the webhook again. n8n must not send a second CAPI event for the same Sale Order.

Steps

1. Create a test Sales Order with custom fields. Confirm CAPI event fires (check Meta Test Events).
2. Edit the Sales Order — change the total amount.
3. Save the edited Sales Order.
4. Check Zoho Books webhook log: did the webhook fire again on the edit?
5. If yes: check n8n execution log — did it attempt to send a second CAPI event?
6. Check Meta Test Events — did a second Purchase event appear?

✓ PASS condition:

```
   Zoho Books does not fire webhook on Sales Order edit — only on creation
   OR: Zoho Books fires on edit but n8n deduplication (T26) prevents second CAPI
event
   Meta Test Events shows only one Purchase event per Sales Order
```

✗ FAIL condition and what it means:

```
   Zoho Books fires on edit AND sends second CAPI event → two options: (1) Change
the Zoho Books Workflow Rule to trigger only on Created not on any update. (2)
Rely on Meta event_id deduplication from T26. Confirm T26 passes first. If T26
also fails, fix T26 before addressing this.
```

T30 — Sales Order deleted

T30	Sales Order deleted — n8n handles gracefully without crashing	Who	Time
		Developer	30 minutes

What we are verifying: When a Sales Order is deleted in Zoho Books (e.g. a test order or an error), n8n handles the incoming webhook gracefully and does not crash the workflow or send a CAPI event for a deleted order.

Steps

1. Create a test Sales Order. Confirm webhook fires to n8n.
2. Delete the Sales Order in Zoho Books.
3. Check Zoho Books webhook log: does a webhook fire on deletion?
4. If yes: check n8n execution log — does the workflow crash or handle gracefully?
5. Confirm no CAPI event is sent for the deleted order.

✓ PASS condition:

```
   Zoho Books does not fire webhook on Sales Order deletion — only on creation
   OR: if webhook fires on deletion, n8n detects missing or zero amount and skips
CAPI event
   No CAPI event sent for deleted order
   n8n workflow does not crash
```

✗ FAIL condition and what it means:

```
   n8n crashes on deletion webhook → add null checks in Workflow 4: if total is
null or 0, stop workflow. Add error handling node so workflow failure is logged
but does not halt the queue.
```

T31 — Meta CAPI fbc format

T31	Meta CAPI fbc parameter is correctly formatted and accepted by Meta	Who	Time
		Developer	30 minutes

What we are verifying: The fbc parameter sent to Meta CAPI is correctly formatted. fbc is the highest-impact field for Event Match Quality — an incorrectly formatted fbc reduces EMQ significantly and weakens Meta's ability to attribute the conversion to the correct ad click.

fbc format specification

fbc format: fb.1.[subdomainIndex].[fbclid]

Example: if fbclid = AbCdEf123456789
Then fbc = fb.1.1721234567890.AbCdEf123456789

Where 1721234567890 is the Unix timestamp (in milliseconds) when the fbclid was first captured.
subdomainIndex is always 1 for standard Shopify stores.

IMPORTANT: Do NOT send fbc if fbclid is empty or null.
An empty or malformed fbc is worse than no fbc — it actively harms EMQ.

Steps

1. In n8n Workflow 4, locate where the fbc value is constructed.
2. Confirm the format is: 'fb.1.' + [timestamp when fbclid was captured] + '.' + fbclid
3. Confirm the timestamp is in milliseconds (13 digits), not seconds (10 digits).
4. Confirm the workflow checks if fbclid is empty before constructing fbc. If fbclid is empty, fbc must be omitted from the payload entirely.
5. Send a test CAPI event with a known fbclid and check the EMQ score in Meta Test Events.
6. Compare EMQ with fbc present vs EMQ without fbc (send same event without fbc). EMQ with fbc should be noticeably higher.

✓ PASS condition:

```
fbc format is fb.1.[13-digit millisecond timestamp].[fbclid]
fbc is omitted when fbclid is empty
EMQ score with fbc present is at least 1.0 point higher than without fbc
```

✗ FAIL condition and what it means:

fbc format wrong → Meta rejects it silently or it contributes nothing to EMQ. Check: timestamp must be milliseconds not seconds (Date.now() not Math.floor(Date.now()/1000)). Subdomain index must be 1.

fbc sent when fbclid is empty → remove fbc from payload when fbclid is null or empty string. An empty fbc corrupts the event.

T32 — Staff bypassing Zendesk

T32	Staff create estimates directly in Zoho Books without going through Zendesk	Who	Time
		Business owner / operations	30 minutes

What we are verifying: If staff skip Zendesk and create estimates directly in Zoho Books, the tracking fields will be empty. The resulting Sales Order will trigger a CAPI event with no attribution data — a wasted event. This test measures how often this happens.

Steps

1. Interview your sales staff directly. Ask:
2. Do you ever open Zoho Books directly to create an estimate without first receiving a Zendesk ticket?
3. How often does this happen? For which brands?
4. What are the circumstances where you go directly to Zoho Books?
5. Review the last 50 estimates in Zoho Books. For each estimate, check: is there a corresponding Zendesk ticket ID in the custom field? If the Zendesk Ticket ID custom field is empty, the estimate was created without going through Zendesk.

Metric	Count	Percentage	Action
Estimates with Zendesk Ticket ID populated			Good — tracked
Estimates WITHOUT Zendesk Ticket ID			Not tracked — CAPI event will have no attribution

✓ PASS condition:

```
Less than 10% of estimates are created without a corresponding Zendesk ticket
Staff confirm they always work through Zendesk before creating an estimate
```

✗ FAIL condition and what it means:

```
More than 10% of estimates bypass Zendesk → implement mandatory process: all
customer inquiries must be received and logged in Zendesk before an estimate is
created. This is a training and management issue. The system cannot fix it
technically — only process discipline can.
Staff say they often go directly to Zoho Books → design a new SOP: always open
Zendesk first, find or create the customer ticket, then create the estimate from
```

```
within that context. Train all brands on this process before go-live.
```

Test results summary — all 32 tests

Record results here after running all tests. Do not start development until all Blocker = Yes tests pass.

Technical tests T1–T12

Test	Description	Result	Blocker?	Action if fail
T1	Session token capture in localStorage	Pass / Fail	Yes	Fix theme.liquid script
T2	Redirect page fires n8n webhook	Pass / Fail	Yes	Debug fetch() call
T3A	WhatsApp phone number in Zendesk	Pass / Fail	No	Time-based matching only
T3B	Viber user ID in Zendesk	Pass / Fail	No	Time-based + lower confidence flag
T3C	Messenger PSID in Zendesk identities	Pass / Fail	No	Time-based matching only
T3D	Email address in Zendesk ticket	Pass / Fail	No	Email address as fallback
T4	Zoho Books payload has custom fields	Pass / Fail	Yes	Add GET API call after webhook
T5	Meta CAPI test event EMQ >= 6.0	Pass / Fail	Yes	Fix payload format or credentials
T6	Token survives redirect page	Pass / Fail	Yes	Fix redirect script localStorage read
T7	Token survives multi-page browsing	Pass / Fail	Yes	Fix token generation check
T8	Email token injection and extraction	Pass / Fail	No	Use email address fallback only
T9	Estimate fields transfer to Sales Order	Pass / Fail	Yes	Add n8n copy step via API
T10	Tag update preserves existing tags	Pass / Fail	Yes	Use /tags endpoint not /tickets/{id}
T11	Multi-brand routing correct	Pass / Fail	Yes	Fix Switch node brand conditions
T12	Safari iOS localStorage persists	Pass / Fail	No	Document as accepted risk

Technical tests T19–T31

Test	Description	Result	Blocker?	Action if fail

T19	Redirect page timing on slow mobile	Pass / Fail	No	Fix fetch keepalive async behavior
T20	Script does not conflict with other apps	Pass / Fail	Yes	Rename localStorage key prefix
T21	Script does not slow page load	Pass / Fail	Yes	Fix script async behavior
T22	Duplicate ticket — same customer two channels	Pass / Fail	Yes	Add estimate deduplication in Workflow 3
T23	Zendesk ticket creation speed for matching	Pass / Fail	Yes	Adjust matching window or confidence flag
T24	RFQ trigger skips agent replies	Pass / Fail	Yes	Add requester filter to trigger condition
T25	n8n downtime and Zoho Books retry	Pass / Fail	Yes	Add polling fallback workflow
T26	No duplicate CAPI events on retry	Pass / Fail	Yes	Add data store deduplication check
T27	Cross-brand phone contamination	Pass / Fail	Yes	Add brand to data store key
T28	Data store TTL expiry works correctly	Pass / Fail	Yes	Fix TTL or add cleanup workflow
T29	Sales Order edit does not duplicate CAPI	Pass / Fail	Yes	Restrict webhook to Created event only
T30	Sales Order deleted — graceful handling	Pass / Fail	No	Add null/zero amount check in Workflow 4
T31	fbc parameter format correct	Pass / Fail	Yes	Fix fbc construction — milliseconds not seconds

Operational tests T13–T18, T32

Test	Description	Result	Blocker?	Action if fail
T13	Staff data capture rate >= 80%	Pass / Fail	Yes	Mandatory SOP + training before go-live
T14	Sales Order created within 24hrs of payment	Pass / Fail	Yes	Staff SOP + confirmation before go-live
T15	RFQ keywords cover real customer language	Pass / Fail	Yes	Update keyword list with all discovered phrases
T16	Channel volume distribution known	Pass / Fail	No	Document actual coverage rate
T17	All brands have distinct contact channels	Pass / Fail	Yes	Each brand must have own

				channels
T18	Staff SOP for untracked inquiries exists	Pass / Fail	Yes	Write and train staff before go-live
T32	Staff not bypassing Zendesk (< 10%)	Pass / Fail	Yes	Mandatory process discipline + training

ALL BLOCKER = YES TESTS MUST PASS BEFORE DEVELOPMENT STARTS. NO EXCEPTIONS.

Non-blockers (T3A, T3B, T3C, T3D, T8, T12, T16, T19, T30): document result and note accepted risk.

PASS threshold for go-ahead: 100% of Blocker = Yes tests pass.

Estimated time to run all 32 tests: 4–5 days.
Estimated time to resolve failures: 1–5 days depending on severity.
Total recommended buffer before development: 2 weeks.